

Solving the Traveling Salesman Problem (TSP) Using Nearest Neighbor Heuristic

Introduction

The Traveling Salesman Problem (TSP) is a well-known combinatorial optimization problem where a salesman must visit a set of cities exactly once and return to the starting city while minimizing travel cost (typically distance). This problem is **NP-hard**, meaning that finding an exact solution for large inputs is computationally expensive.

In this implementation, we use the **Nearest Neighbor Heuristic**, a greedy algorithm that constructs a tour by repeatedly visiting the closest unvisited city.

Algorithm Description

1. Read Input:

- The first line specifies the TSP type (not used in computation).
- The second line provides the number of cities `NNN`.
- The next `NNN` lines contain the coordinates of each city (not used in the algorithm, only for visualization).
- The last `NNN` lines contain an $N \times NN \times NN \times N$ **distance matrix**, where `distance_matrix[i][j]` represents the distance from city `iii` to city `jjj`.

2. Nearest Neighbor Heuristic:

- Start from **city 0**.
 - Repeatedly select the **closest unvisited city** as the next stop.
 - Continue until all cities are visited.
 - Output the tour in the order of visited cities.
-

Code Explanation

1. Reading Input (`read_input` function)

The function reads the problem type, number of cities, their coordinates, and the distance matrix from standard input.

```
def read_input():
    tsp_type = sys.stdin.readline().strip() # Read TSP type
    N = int(sys.stdin.readline().strip()) # Number of cities

    # Read coordinates (not used in computation)
    coordinates = [tuple(map(float, sys.stdin.readline().split())) for
_ in range(N)]

    # Read distance matrix
    distance_matrix = [list(map(float, sys.stdin.readline().split()))
for _ in range(N)]

    return tsp_type, N, coordinates, distance_matrix
```

2. Nearest Neighbor TSP Solver (**tsp_nearest_neighbor** function)

This function constructs a tour using the Nearest Neighbor heuristic:

```
def tsp_nearest_neighbor(N, distance_matrix):
    """ Solves TSP using Nearest Neighbor Heuristic """
    unvisited = set(range(1, N)) # Set of unvisited cities
    tour = [0] # Start from city 0

    while unvisited:
        last = tour[-1] # Current city
        # Choose the nearest unvisited city
        next_city = min(unvisited, key=lambda city:
distance_matrix[last][city])
        tour.append(next_city)
        unvisited.remove(next_city)

    return tour
```

- A **set** is used to keep track of unvisited cities.
- The algorithm iteratively selects the nearest **unvisited** city and adds it to the tour.
- The process continues until all cities are visited.

3. Solving the TSP (**solve_tsp** function)

This function orchestrates the input reading, algorithm execution, and result output.

```
def solve_tsp():
    tsp_type, N, coordinates, distance_matrix = read_input()
    best_tour = tsp_nearest_neighbor(N, distance_matrix)
    print(" ".join(map(str, best_tour))) # Output the computed tour
```

4. Running the Program (**__main__** block)

```
if __name__ == "__main__":
    solve_tsp()
```

This ensures the script runs properly when executed.

Time Complexity Analysis

- **Nearest Neighbor Heuristic:** $O(N^2)$
 - Finding the nearest city takes $O(N)$.
 - This is repeated N times, leading to an overall complexity of $O(N^2)$.
- **Space Complexity:** $O(N^2)$ (due to storing the distance matrix).